

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0611

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 2

Total Marks:20

Annexure A

Coding Challenge

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

CODE:

```
import numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Example dataset (features + labels)
X = np.array([
    [2, 4], [4, 4], [4, 6], [6, 2], [6, 4],
    [8, 4], [8, 6], [10, 2], [10, 4], [10, 6]
])
y = np.array([0, 0, 0, 1, 1, 1, 1, 2, 2, 2]) # class labels

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Euclidean distance function
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

# KNN classifier
def knn_predict(X_train, y_train, x_test, k):
    distances = [euclidean_distance(x_test, x) for x in X_train]
    k_indices = np.argsort(distances)[:k]
    k_labels = [y_train[i] for i in k_indices]
    return Counter(k_labels).most_common(1)[0][0]

# User input for K
k = int(input("Enter value of K: "))

# Predictions
y_pred = [knn_predict(X_train, y_train, x, k) for x in X_test]

# Results
print("Predicted labels:", y_pred)
print("Actual labels   :", list(y_test))
print("Accuracy       :", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
Enter value of K: 45
Predicted labels: [np.int64(1), np.int64(1), np.int64(1)]
Actual labels   : [np.int64(2), np.int64(0), np.int64(1)]
Accuracy        : 0.3333333333333333
```

9. Create a Python program to implement the **K-Means clustering algorithm**. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generate synthetic dataset
X, _ = make_blobs(n_samples=200, centers=4, cluster_std=1.0, random_state=42)

# K-Means implementation
def kmeans(X, k, max_iters=100):
    # Randomly initialize centroids
    np.random.seed(42)
    centroids = X[np.random.choice(X.shape[0], k, replace=False)]

    for _ in range(max_iters):
        # Assign points to nearest centroid
        distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
        labels = np.argmin(distances, axis=1)

        # Update centroids
        new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])

        # Stop if centroids don't change
        if np.allclose(centroids, new_centroids):
            break
        centroids = new_centroids

    return labels, centroids

# User input for number of clusters
k = int(input("Enter number of clusters (K): "))

labels, centroids = kmeans(X, k)

# Visualization
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis", s=30)
plt.scatter(centroids[:, 0], centroids[:, 1], c="red", marker="X", s=200)
```

```
plt.title(f'K-Means Clustering (k={k})')  
plt.show()
```

OUTPUT:

Enter number of clusters (K): 55

